



The Iby and Aladar Fleischman
Faculty of Engineering
Tel Aviv University



SyncRun - Align Music to Run

Project Number: 25-1-1-3408

Presenters:

Matan Granot & Yogev Grinblatt

Advisor:

Dr. Lior Arbel



Project Overview

- SyncRun dynamically adjusts the playback rate of a rhythmic music track to seamlessly match a runner's offline step rhythm.
- It uses advanced time-stretch algorithms to synchronize musical beats with physical steps, without altering the original pitch.



Motivation



Traditional fitness music applications force runners to adapt to a fixed, pre-set tempo.



We want to reverse this dynamic by making the music adapt to the runner's fluctuating pace.



Seamless rhythm synchronization significantly improves workout flow, motivation, and physical coordination.



Problems

BPM Mismatch

The natural beats per minute (BPM) of most standard songs are significantly slower than an average runner's cadence.

The "Chipmunk" Effect

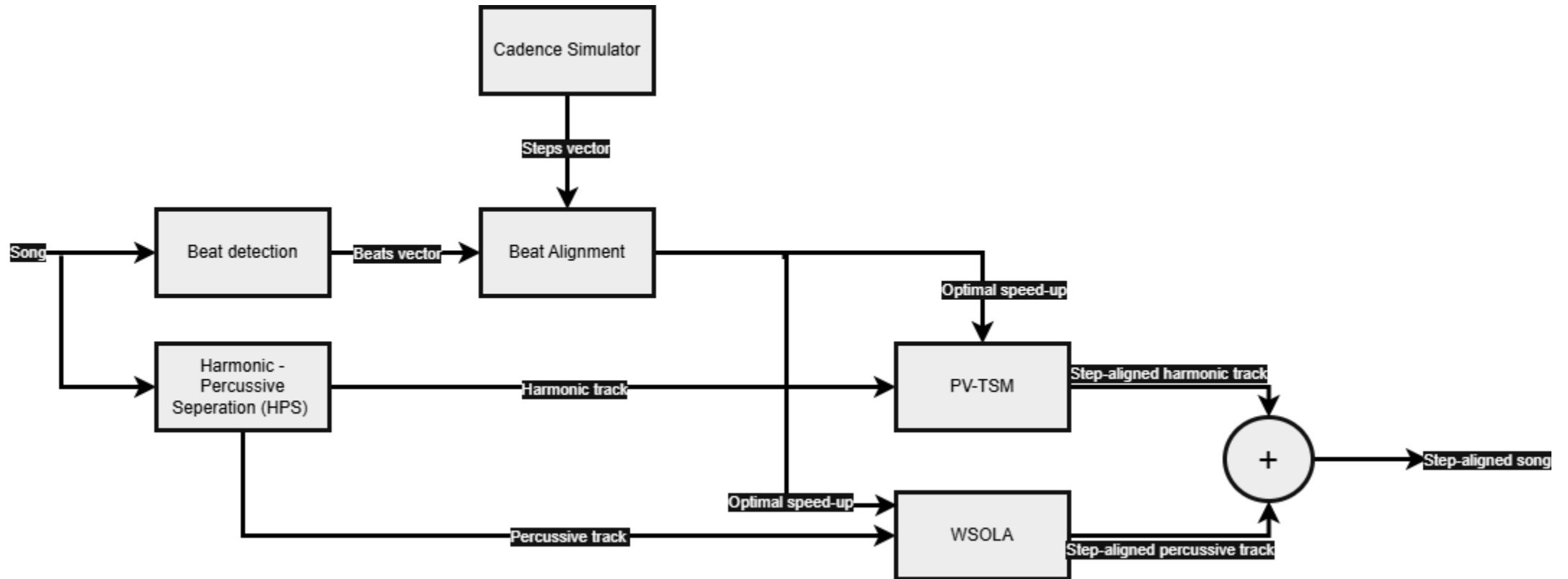
Simply speeding up a song mathematically causes severe pitch distortions and smearing of transient drum hits.

Flawless Synchronization

The challenge is aligning fluctuating physical rhythms with musical beats offline, completely free of audio artifacts.



Block Diagram



Beat Detection

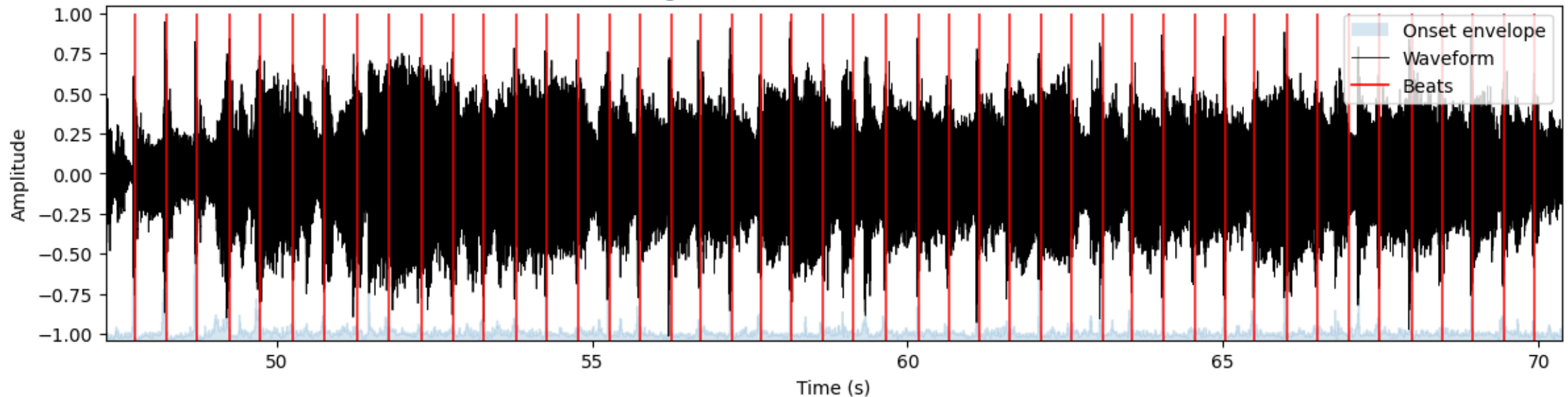


We rely on **librosa** to perform offline onset detection to extract the song's overall tempo and exact beat timestamps.



Input song + clicks on beat

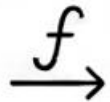
Song Waveform - BPM = [123.046875]



Beat Alignment



$$\operatorname{argmin}_{r \in (0.8, 1.5)} |\overline{song} - r * \overline{cadence}|$$



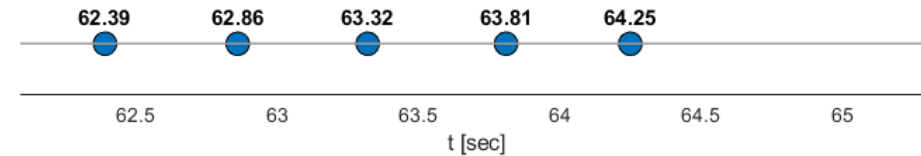
To find the optimal playback speed, we minimize the absolute L1 distance between the synthesized runner steps and the song's beats.



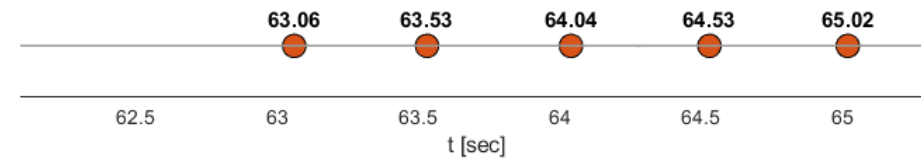
A weighted median optimizer computes the perfect scaling ratio for every discrete 2.5-second segment.

Beat Alignment

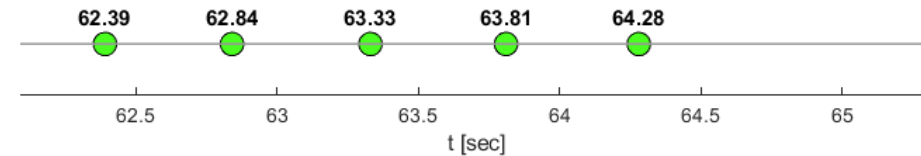
Run Steps



Song Beats (before alignment)



Song Beats (after x1.01 speed up)

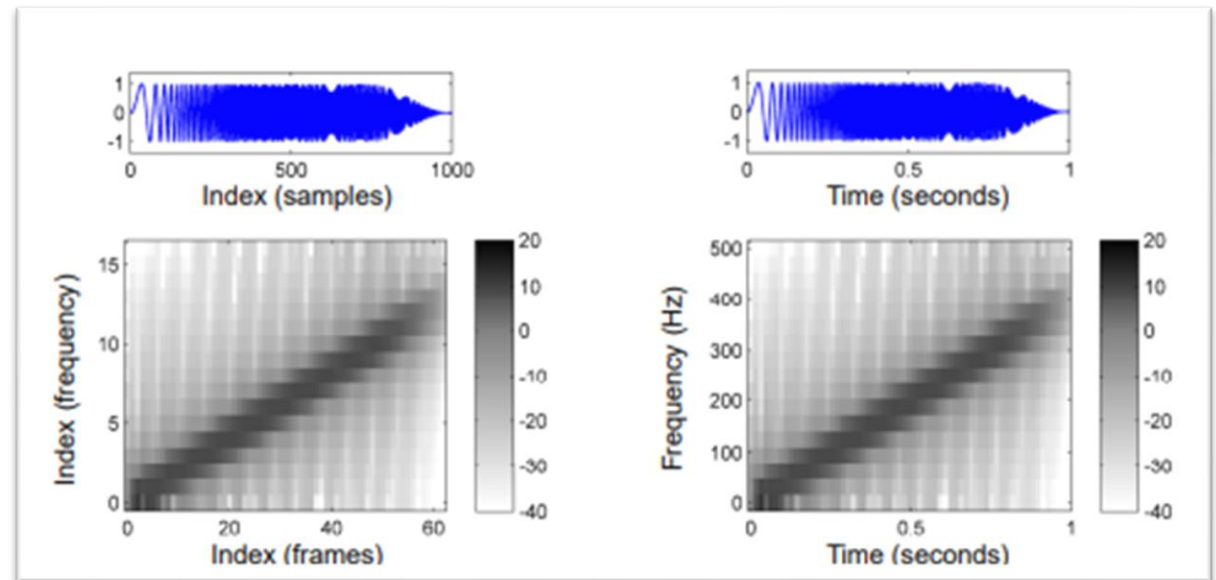
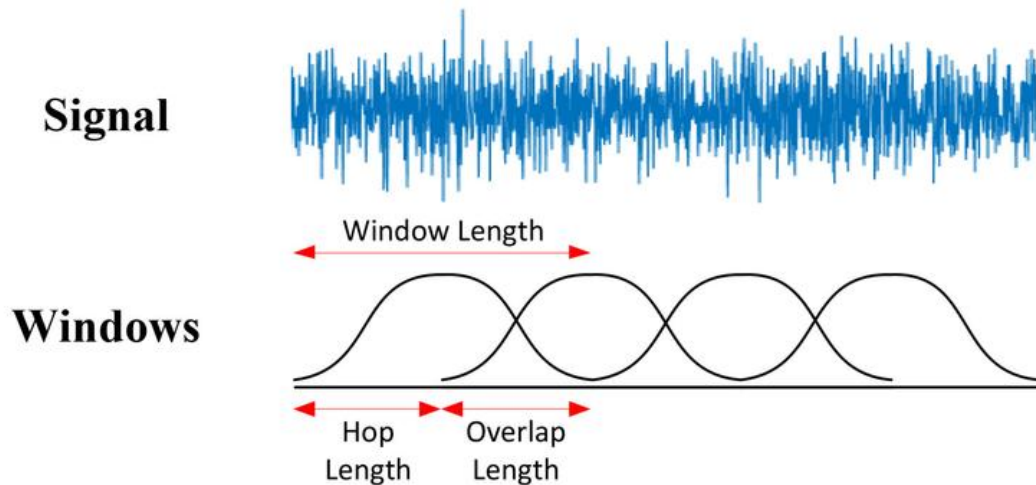




Our main weapon: Short-Time Fourier Transform (STFT)

- The audio is first transformed from the time domain into the frequency domain. We utilize a window size of 2048 samples with a 75% overlap to ensure smooth reconstruction and phase coherence.

$$X(m, k) = \sum_{r=-N/2}^{N/2-1} x_m(r)w(r) \exp\left(-\frac{2\pi ikr}{N}\right)$$





Harmonic – Percussive separate (HPS)

- We applied median filters to separate the original audio into its sustained, pitched sounds (harmonic content) and sharp, transient drum hits (percussive content)

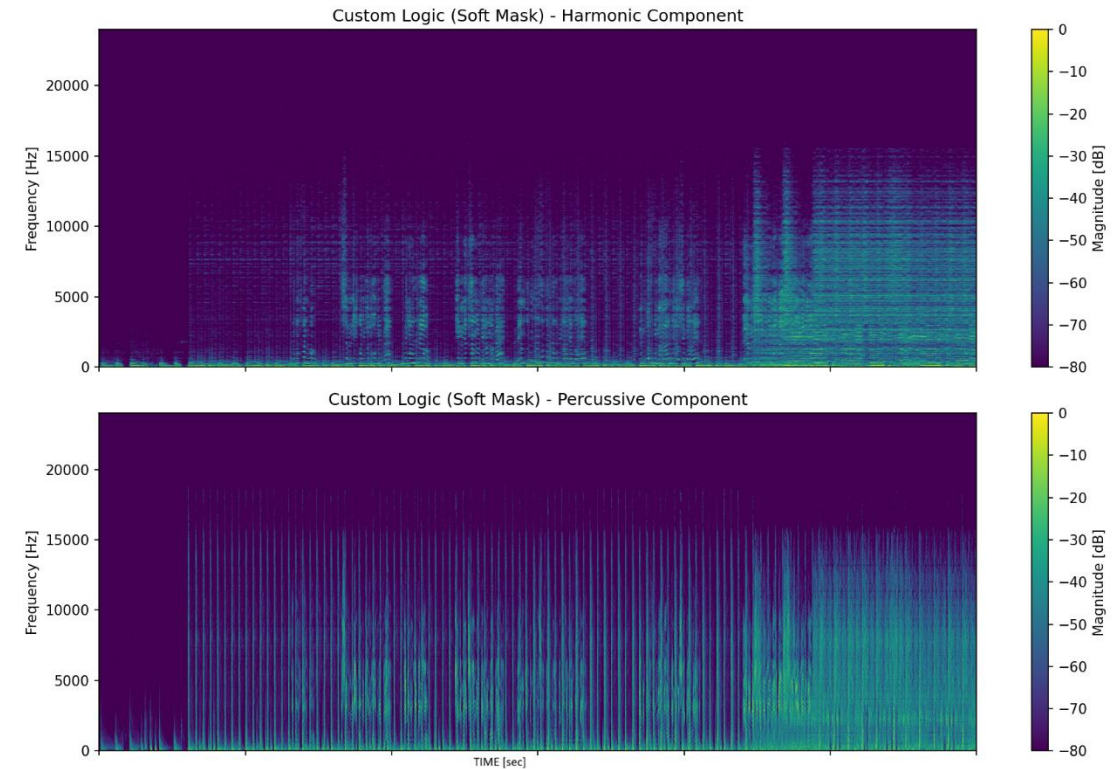
$$M_h(m, k) = \begin{cases} 1, & \text{if } \tilde{Y}_h(m, k) > \tilde{Y}_p(m, k) \\ 0, & \text{otherwise} \end{cases}$$

$$M_p(m, k) = \begin{cases} 1, & \text{if } \tilde{Y}_p(m, k) \geq \tilde{Y}_h(m, k) \\ 0, & \text{otherwise} \end{cases}$$

- And created binary masks are applied pointwise to the original complex STFT $X(m, k)$ to isolate the complex frequency representations of each component

$$X_h(m, k) = X(m, k) \cdot M_h(m, k)$$

$$X_p(m, k) = X(m, k) \cdot M_p(m, k)$$





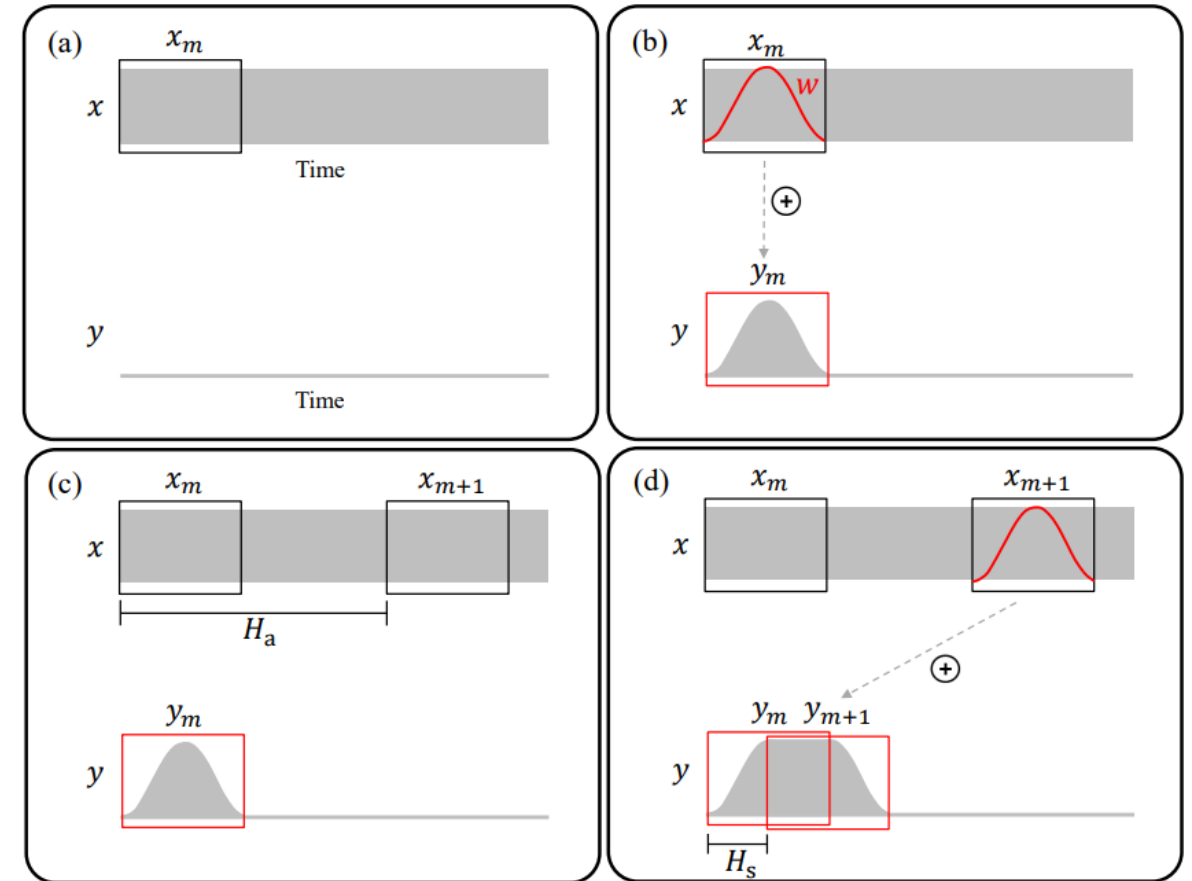
OLA – good, but not for us

Naive TSM: Standard Overlap-Add (OLA)

Overlap-Add (OLA) is a fundamental, naive **time-domain** technique that segments audio into equal, overlapping frames.

Instead, it sums these frames directly without alignment, causing noticeable **phase jumps** and related artifacts.

$$r_{\text{speed_up}} = \frac{H_a}{H_s}$$





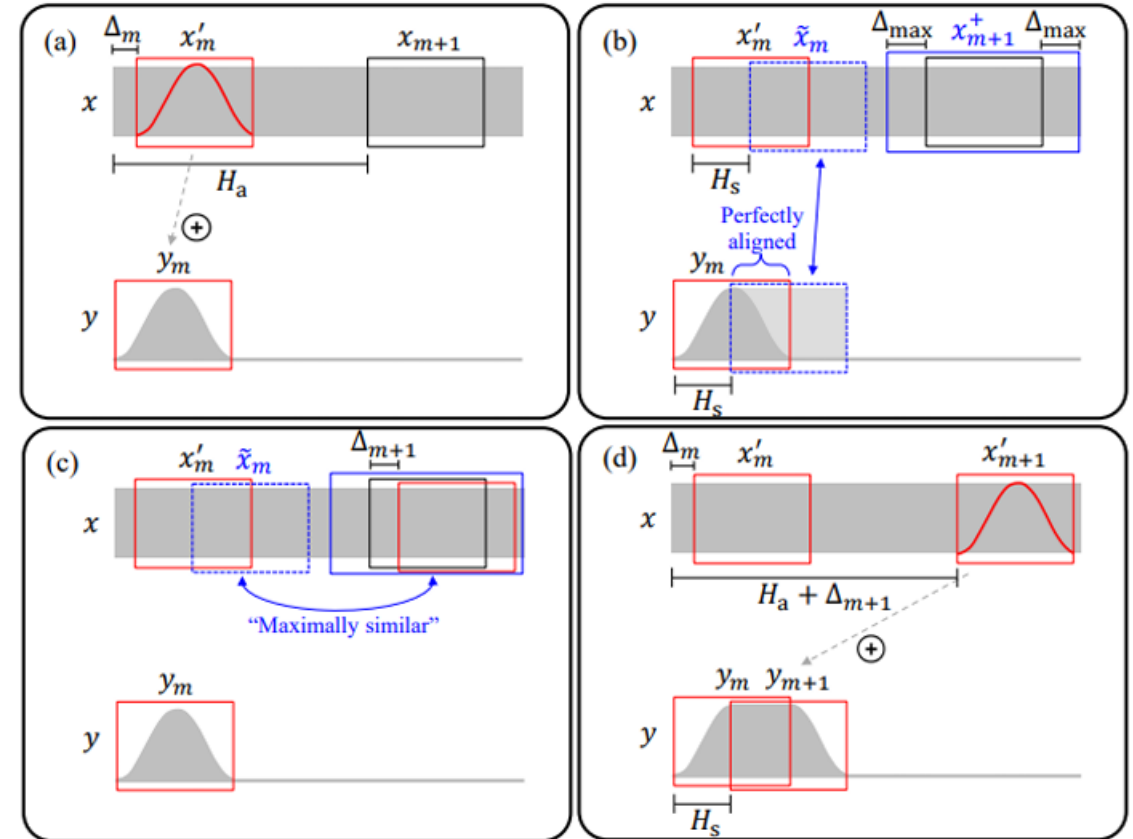
WSOLA

Applied to: The Percussive Component

Waveform Similarity Overlap-Add (WSOLA) is a **time-domain** technique that stretches audio without phase jump artifacts.

It preserves the punch of drum hits by aligning adjacent frames along their periodic peaks.

$$\tilde{x}_m = \operatorname{argmax}_{x \in \mathcal{X}} (\text{Cross-Correlation}(x'_m, x))$$





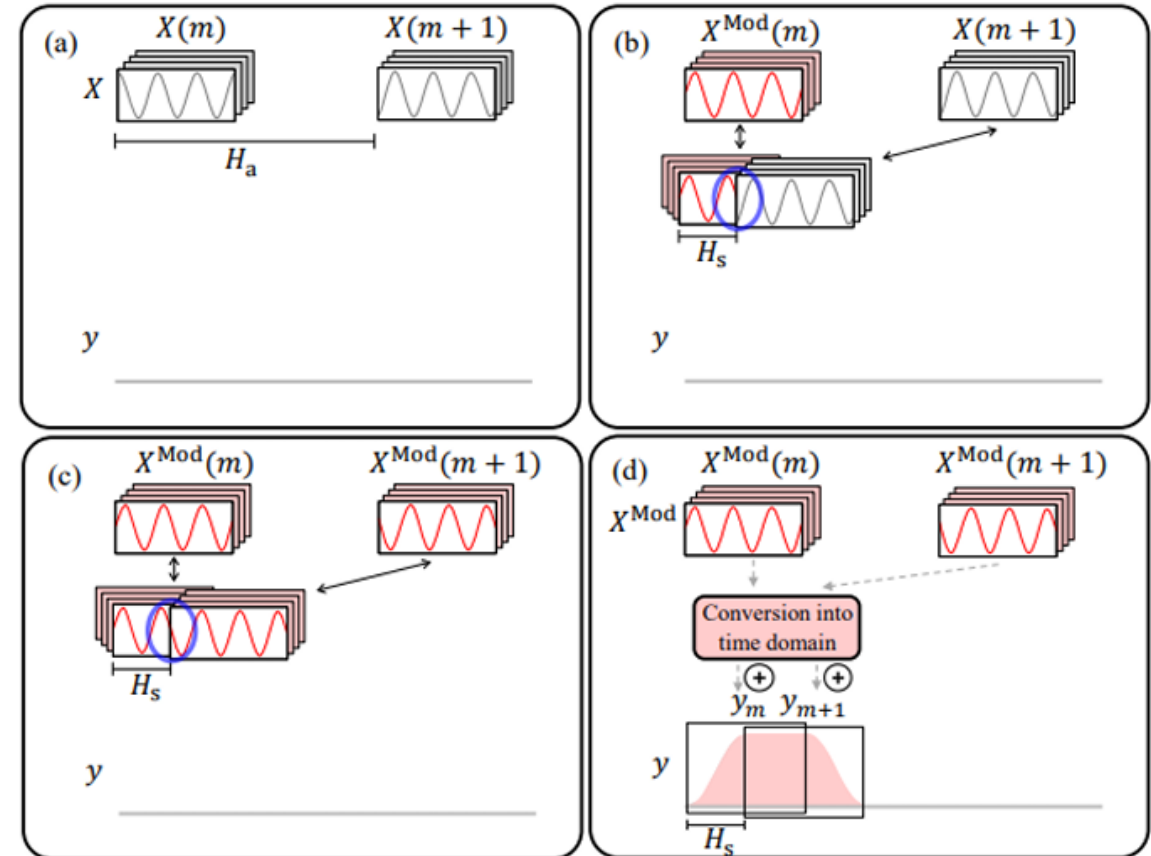
Phase Vocoder TSM

Applied to: The Harmonic Component

- This frequency-domain technique updates frame phases recursively to maintain horizontal coherence across stretched sinusoids.
- It allows us to stretch vocals and basslines perfectly without altering their pitch.

$$\theta(m, k) = \theta(m - 1, k) + H_s \cdot \omega(m, k)$$

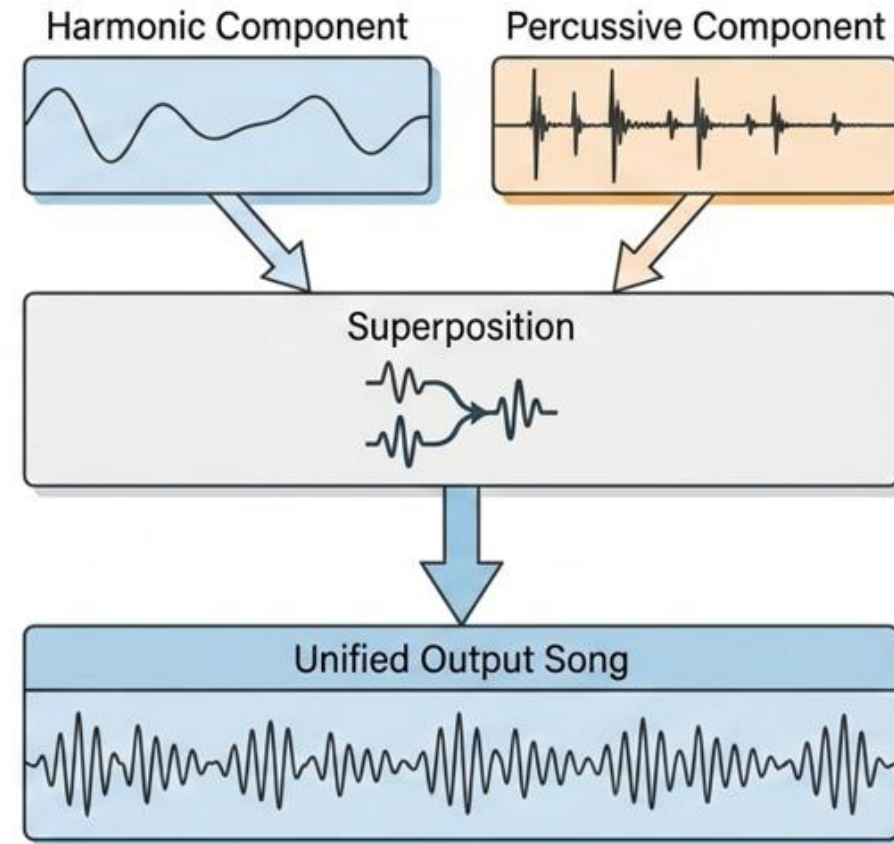
$$X^{\text{Mod}}(m, k) = |X(m, k)| \exp(i\theta(m, k))$$





Reconstruction

- The modified harmonic and percussive matrices are inverted back to the time domain.
- The algorithm dynamically trims all components to the exact same minimum length.
- The tracks are aggregated via superposition into a single, synchronized, click-free output song.





System Evaluation: Alignment & Scaling

System Requirement	Validation Result
Input Compatibility: Capable of processing signals differing by $< 10\%$.	Our L1 Weighted Median Optimizer successfully converged without diverging in all tests.
Speed Ratio Range: Must operate stably between $0.8x$ and $1.5x$.	Strictly tuned engine parameters handled aggressive synthetic drift stress-tests flawlessly.
Beat Alignment Accuracy: Phase offset must be $< 50ms$.	Achieved a highly accurate average alignment error of $15.67ms$.

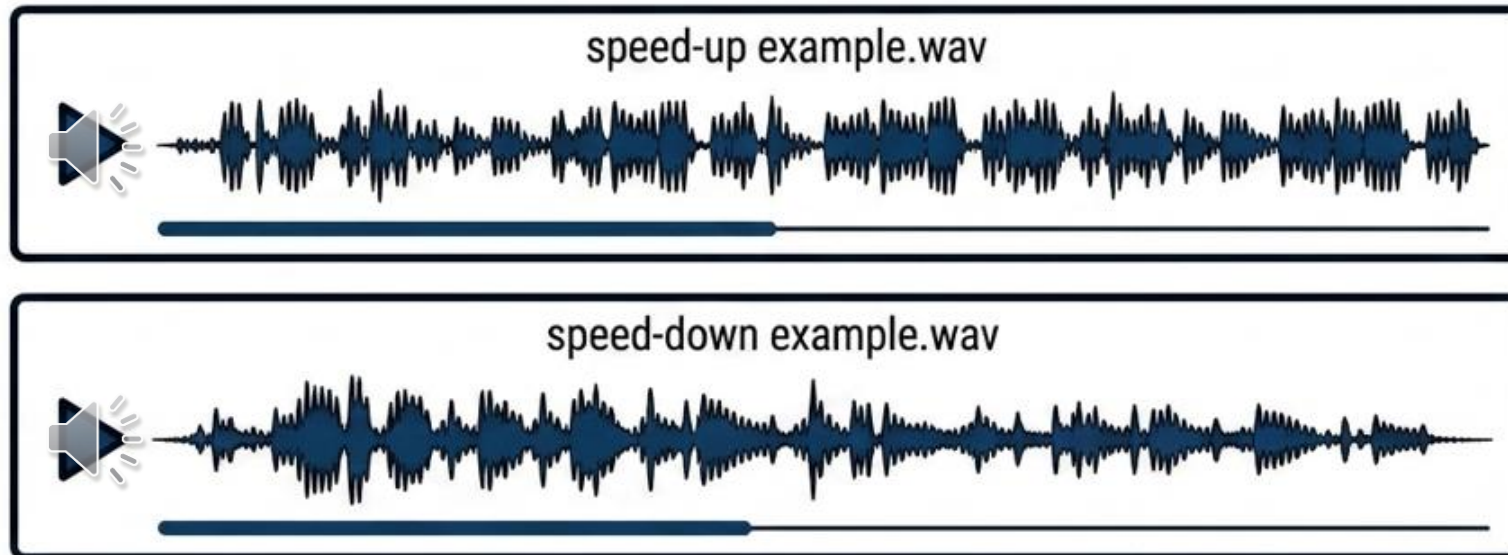


System Evaluation: Speed & Fidelity

System Requirement	Validation Result
Processing Latency: Must stay below 2 seconds per segment.	The pipeline achieved a computational latency of only 475ms per 2.5-second chunk.
Audio Quality: Sampling rate must exceed 20 kHz.	The system strictly processes and exports lossless .wav audio at a high-fidelity 44.1 kHz.



Listen to the real-time pitch-preserving
time-scale modification.

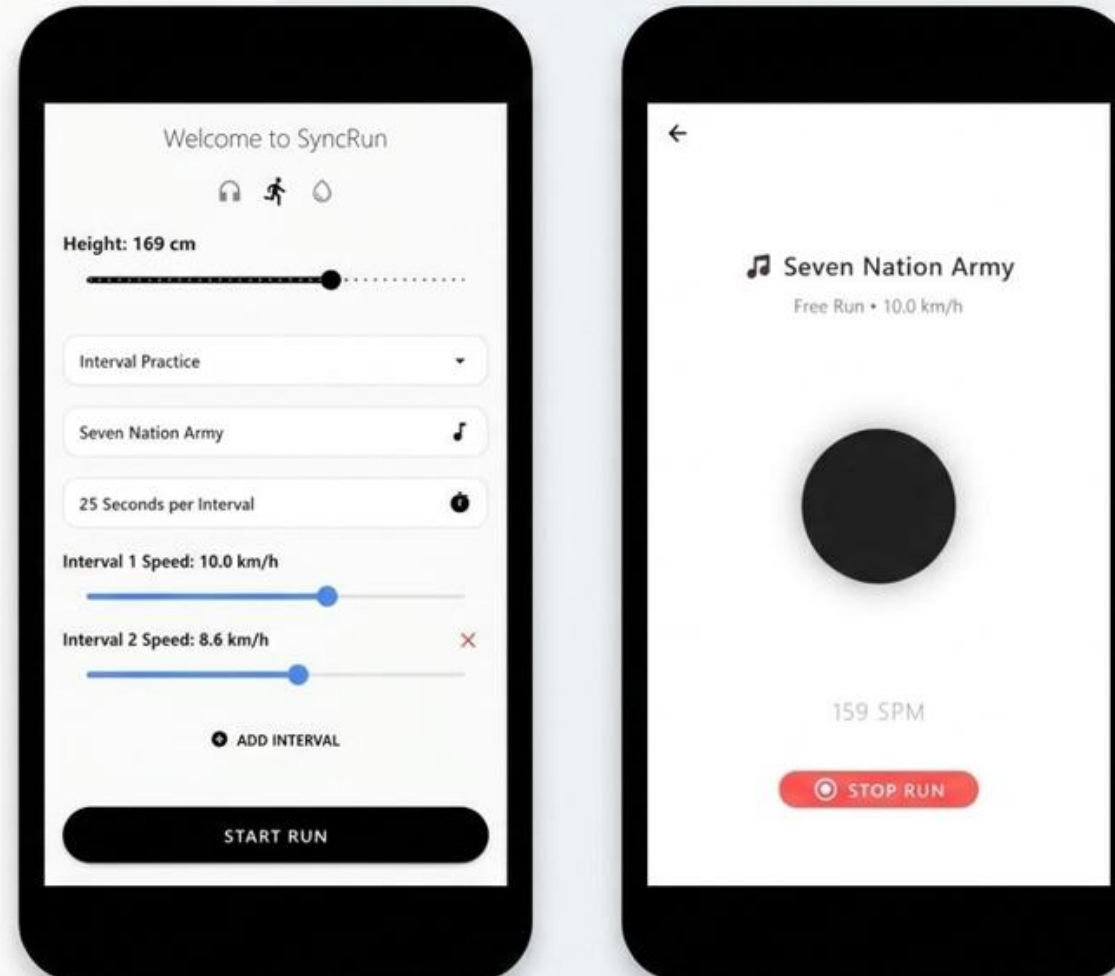




The Mobile Interface

We transitioned the complex Python backend into a user-friendly Flutter mobile application.

Users can define physical metrics, select songs, and manually assign dynamic speed intervals.





Future Work

- **Transition to real-time, adaptive tempo adjustments using live biometric feedback from smartwatches, accelerometers, and barometers.**
- **Shift processing directly to the mobile device and integrate with commercial platforms like Spotify to synchronize personalized playlists on the fly.**



Thank you for listening, questions?

SyncRun